

LUCRAREA 7

MICROPROCESORUL 8086 (II)

1. OBIECTUL LUCRĂRII

Lucrarea continuă studiul microprocesorului 8086, atât din punct de vedere hardware cât și software și prezintă aplicații cu acest microprocesor.

2. BREVIAR TEORETIC

Microprocesorul 8086 dispune de un mecanism special de preluare a instrucțiunilor ce urmează a fi executate. El constă dintr-o memorie de tip FIFO (coadă de instrucțiuni - queue instruction), memorie care are 6 octeți (locații). Această memorie internă este umplută de mecanismele microprocesorului, pe perioada în care are loc execuția unei instrucțiuni anterioare, perioadă în care magistralele microprocesorului nu sunt folosite. Coada de instrucțiuni permite creșterea vitezei de lucru a microprocesorului, practic desfășurându-se mai multe activități în paralel.

Procesul de paralelism se oprește însă, la extragerea unor noi coduri de instrucțiuni ce se vor executa, fără să aibă loc în paralel și derularea execuției acestor instrucțiuni.

Există microprocesoare care pot executa simultan mai multe instrucțiuni.

I1 :

F	D	AO	EX
---	---	----	----

I2 :

F	D	AO	EX
---	---	----	----

I3 :

F	D	AO	EX
---	---	----	----

F = fetch

D = decodificare

AO = aducere operand

EX = execuție

Instrucțiunile aduse în coada de instrucțiuni nu sunt deloc analizate de microprocesor. Ele sunt preluate din memorie prin simplul fapt că se află în spatele instrucțiunii ce se execută curent. Lucrul se bazează pe observația că, de obicei, în cadrul unui program instrucțiunile ce urmează a fi executate sunt plasate liniar una după alta.

În situații speciale - cum sunt instrucțiunile de salt, apelurile de subrutine - conținutul cozii de instrucțiuni este resetat (șters) de către procesor.

Registrul IR (instruction register) este registrul în care este stocată instrucțiunea ce urmează a fi executată de către procesor, pe toată perioada ei de execuție.

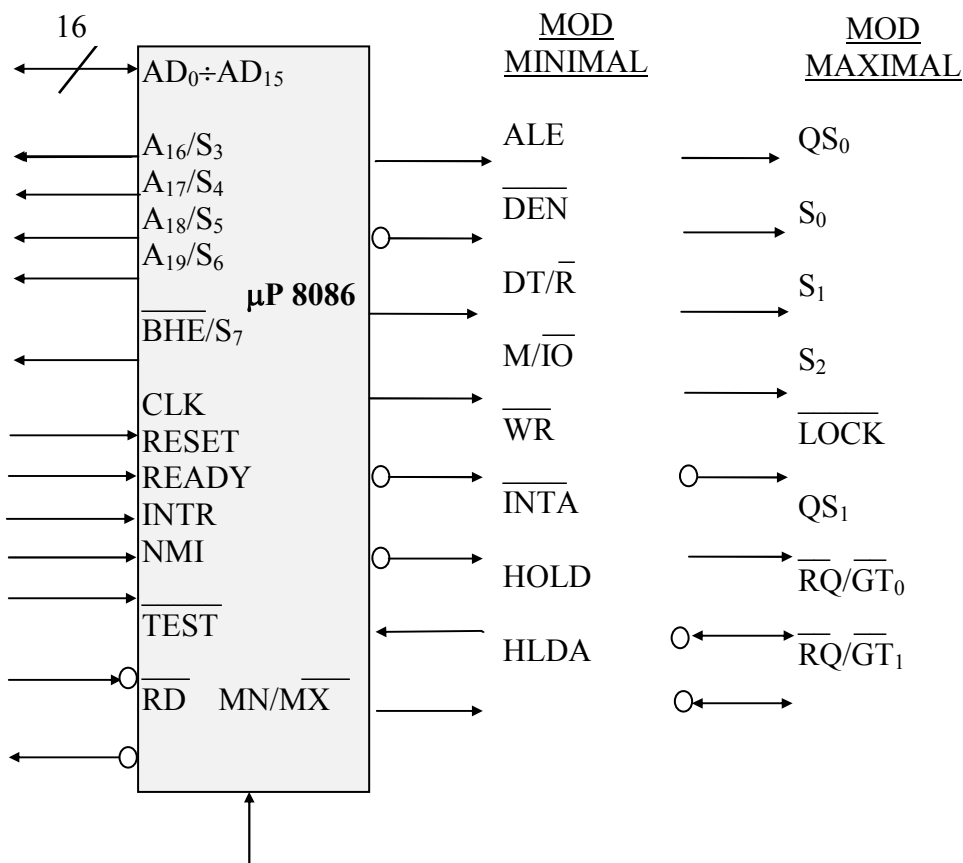
Îndrumar de laborator 7

Blocul de control este alcătuit dintr-un automat de tip CROM acționat de semnalul extern de ceas (CLK). Acest bloc decodifică instrucțiunea primită și ia măsuri pentru execuție. De asemenea, el controlează întreaga activitate a microprocesorului.

Sunt mai multe categorii de pini ai μP 8086:

- există o categorie de pini care indiferent de modul de lucru (minimal sau maximal) au aceeași semnificație (8 pini);
- o a doua categorie de pini sunt de tipul multiplexat, cu păstrarea aceleiași semnificații indiferent de modul de lucru. Pe acești pini se vehiculează informații diferite în cadrul unui ciclu mașină, la derularea unei instrucțiuni (21 pini);
- o a treia categorie de pini au semnificație diferită în funcție de modul de lucru al microprocesorului (8 pini);
- o a patra categorie o constituie pinii rezervați alimentării: 1 pin (+VCC), 2 pini (GND).

Dimensiunea chip-ului este de 37mm^2 și conține 29000 tranzistoare.



SEMNALE CU ACEIAȘI SEMNIFICAȚIE

$\overline{MN}/MX$ = pinul prin care se alege modul de lucru al microprocesorului (minimal/maximal).

CLK = intrarea prin care se aplică semnalul de ceas. Referitor la semnalul acceptat de microprocesor pentru această intrare, nivelul de '1' este de 3,5 V, iar factorul de umplere al acestuia este 1/3. Un astfel de semnal este generat de circuitul 8284 (generator de ceas).

RESET = semnal de inițializare al microprocesorului; este activ "1" - logic. Pentru a fi luat în considerare de microprocesor trebuie să rămână activ minim 10 perioade ale semnalului de ceas aplicat.

Acțiunile derulate de procesor la RESET sunt :

- registrul CS se încarcă cu constanta $FFFF_{(H)}$;

- registrul IP se încarcă cu constanta $0000_{(H)}$.

De aici rezultă ca prima instrucțiune pe care microprocesorul o execută se află la adresa fizică de memorie $FFFF0_{(H)}$; practic până la sfârșitul memoriei ($FFFFF_{(H)}$) mai rămân 16 locații de memorie. Programatorul trebuie să execute în acest spațiu restrâns, în mod obligatoriu un salt către o zonă liberă de memorie în care va putea scrie programul propriu-zis de lucru.

READY = intrare folosită pentru asigurarea sincronizării funcționării procesorului cu memoria sau circuitele I/O lente. Dacă intrarea READY = '1' - logic, nu se introduc stări de așteptare în derularea instrucțiunilor; acestea apar când READY = '0' - logic. Practic semnalul este echivalent cu WAIT de la $\mu P Z-80$.

Semnalele RESET și READY sunt generate de circuitul generator de ceas - 8284 și sunt sincrone microprocesorului în raport cu semnalul de ceas.

INTR = interrupt - intrare pe care se aplică cererile de întreruperi mascabile (activă "1"- logic).

NMI = intrare pe care se aplică cererile de întreruperi nemascabile (activă "1"- logic).

TEST = intrare folosită la rularea instrucțiunii WAIT. Execuția acestei instrucțiuni permite sincronizarea derulării activității microprocesorului cu evenimentele externe care survin. Astfel, la executarea instrucțiunii WAIT:

- dacă intrarea TEST este '0', microprocesorul va continua normal cu execuția următoarei instrucțiuni;

- dacă intrarea TEST este '1', microprocesorul intră într-o stare de așteptare software, netrecând la execuția următoarei instrucțiuni decât dacă intrarea TEST este '0' - logic.

$\overline{RD}$ = ieșire prin care microprocesorul semnalizează momentele în care realizează o citire pe magistrala de date (fie de la memorie, fie de la circuitele I/O).

SEMNALUL CU CARACTER MULTIPLEXAT

$AD_0 \div AD_{15}$ = linii ce constituie magistrala multiplexată de adrese și date.

A_{16} / S_3 , A_{17} / S_4 , A_{18} / S_5 , A_{19} / S_6 = ieșiri multiplexate prin care se emite partea c.m.s. a adresei fizice și, respectiv, informația de stare despre microprocesor.

$\overline{BHE} / S_7$ = byte high enable (octetul superior valid) - ieșire cu caracter multiplexat prin care microprocesorul indică momentele în care se transmite octetul superior al informației de date și, respectiv, semnalul de stare S_7 .

SEMNALUL CU SEMNIFICAȚIE DIFERITĂ FUNCȚIE DE MODUL DE LUCRU

MODUL MINIMAL

$\overline{ALE}$ = address latch enable; ieșire care validează latch-urile pentru adrese sau celulele de memorie pentru adrese. Este un semnal care permite în exterior demultiplexarea liniilor de adrese, precum și a semnalului $\overline{BHE}$, semnal transmis multiplexat de către procesor. Semnalul este folosit pentru asigurarea memorării acestor informații în latch-uri externe microprocesorului, adică în circuite de tipul 8282.

$\overline{DEN}$ = data enable; ieșire care semnalizează momentele în care microprocesorul vehiculează informații pe magistrala de date. Semnalul este folosit pentru validarea buffer-ilor bidirecționali 8286 (ce formează magistrala de date).

$\overline{DT/R}$ = data transmitted/received; semnal cu caracter de ieșire ce semnalizează sensul transmiterii informației pe magistrala de date:

- dacă este '1' - logic, datele sunt transmise de procesor (scrise);

- dacă este '0' - logic, datele sunt recepționate de procesor (citite).

Semnalul se aplică bufferilor 8286 în vederea precizării direcționalității acestora.

$\overline{M/IO}$ = ieșire ce semnalizează spațiul de adrese cu care μP lucrează:

- dacă este '1' - logic, μP -ul lucrează cu memoria;

- dacă este '0' - logic, μP -ul lucrează cu circuitele I/O (spațiul de adrese I/O este de 64 Kocteti)

$\overline{WR}$ = ieșire ce semnalizează scrierile efectuate de μP pe magistrala de date.

$\overline{INTA}$ = interrupt acknowledge; ieșire ce indică luarea la cunoștință a stării de întrerupere; prin ea, μP -ul semnalizează, după primirea cererilor de întrerupere mascabile, începerea procesului de tratare al acestora; semnalul se adresează de obicei, unor circuite specializate I/O (circuitul 8259A - PIC - programmable interrupt controller); este generat în vederea preluării de la acest circuit, a vectorului de întrerupere prin care se obțin informații relative la adresa subrutinei ce tratează întreruperea.

Îndrumar de laborator 7

HOLD = intrare prin care se poate solicita microprocesorului, cedarea controlului magistralelor (trecerea ieșirilor în starea de înaltă impedanță). Cererea este activă '1'- logic.

HLDA = hold acknowledge; ieșire ce reprezintă răspunsul microprocesorului la cererea HOLD făcută anterior. Această ieșire trece în '1'- logic, după bascularea ieșirilor microprocesorului în starea 'three state', ca urmare a unei cereri HOLD. Dacă HOLD = '0'- logic, rezultă HLDA = '0'- logic.

MODUL MAXIMAL

S_0, S_1, S_2 = semnale de stare prin care este codificată activitatea derulată de microprocesor.

Semnalele se oferă circuitului 8288 în vederea decodificării și a formării semnalelor ce constituie magistrala de comenzi în modul maximal.

Codarea este făcută conform următorului tabel :

S0	S1	S2	STARE 8086	IEȘIREA ACTIVATĂ LA 8288
0	0	0	INTA	INTA
0	0	1	I/O READ	IORC
0	1	0	I/O WRITE	IOWC ; AIOWC
0	1	1	HALT	-----
1	0	0	FETCH	MRDC
1	0	1	MEM READ	MRDC
1	1	0	MEM WRITE	MWTC ; AMWTC
1	1	1	NO OP.	-----

QS_0, QS_1 = queue status; ieșiri ce indică starea cozii de așteptare. Instrucțiunile microprocesorului 8086 sunt codate pe 1 până la 6 octeți.

Codarea este următoarea :

QS1	QS0	SEMNIFICAȚIE
0	0	NO OP.
0	1	- s-a extras primul octet al unei instrucțiuni din coada de așteptare
1	0	- coada de așteptare este goală
1	1	- extragerea unui alt octet decât primul din coada de așteptare.

$\overline{LOCK}$ = lacăt (zavor); este un semnal ce se activează în urma executării unei instrucțiuni specializate numite LOCK. Prin activarea LOCK se semnalizează dispozitivelor externe ce ar putea solicita magistralele

Îndrumar de laborator 7

microprocesorului să nu emită astfel de cereri pe timpul executării instrucțiunii următoare.

$\overline{RQ/GT0}$, $\overline{RQ/GT1}$ = semnale cu caracter bidirecțional, independente, prin care alte microprocesoare sau coprocesoare solicită cedarea controlului magistralelor procesorului 8086.

Setul de regiștri ai microprocesorului 8086 este format din 8 regiștri generali de câte 8 biți (AH, AL, BH, BL, CH, CL, DH, DL). Acești regiștri se pot concatena doi câte doi (de ex. AH + AL $\Rightarrow$ AX), formând regiștri extinși pe 16 biți (AX, BX, CX, DX). Mai există și 4 regiștri pe câte 16 biți:

- 2 regiștri index
 - SI = source index
 - DI = destination index
- 2 regiștri pointeri (indicatori)
 - BP = base pointer
 - SP = stack pointer

Regiștri SI și DI se folosesc împreună cu DS și ES în vederea adresării datelor.

Regiștri BP și SP se folosesc pentru adresarea stivei împreună cu SS.

Registrul AX este acumulatorul microprocesorului 8086 (pe 16 biți) - AH = A high, AL = A low. Acest registru nu mai este un "loc îngust", cum se întâmplă la μP Z-80. Astfel, μP 8086 poate să execute operații aritmetico-logice fără ca unul din operanzi să fie adus în registrul acumulator. Prin setul de instrucțiuni există posibilitatea efectuării instrucțiunilor fie cu regiștri extinși pe 16 biți, fie cu jumătățile acestor regiștri. În setul de instrucțiuni există instrucțiuni pe 8, 16 sau 32 biți.

INSTRUCȚIUNI LOGICE, DE DEPLASARE ȘI ROTAȚIE

Aceste instrucțiuni operează la nivel de rang; operațiile logice se fac între rangurile corespunzătoare ale operanzilor, iar celelalte modifică operandul prin deplasare, respectiv, rotație cu unul sau mai multe ranguri.

INSTRUCȚIUNI LOGICE

1. AND D, S (And)

Instrucțiunea execută operația SI logic între rangurile corespunzătoare operanzilor sursă și destinație.

Operație efectuată:

$$D \leftarrow S \wedge D$$

Indicatori afectați:

$$SF, ZF, PF, CF \leftarrow 0, OF \leftarrow 0.$$

DESTINAȚIE	SURSĂ
registru	registru
registru	memorie
memorie	registru
registru	imediat
memorie	imediat
acumulator	imediat

2. OR D, S (Or)

Instrucțiunea execută operația SAU logic între rangurile corespunzătoare operanzilor sursă și destinație.

Operație efectuată:

$$D \leftarrow S \vee D$$

Indicatori afectați:

$$SF, ZF, PF, CF \leftarrow 0, OF \leftarrow 0.$$

DESTINAȚIE	SURSĂ
registru	registru
registru	memorie
memorie	registru
registru	imediat
memorie	imediat
acumulator	imediat

3. XOR D, S (Xor)

Instrucțiunea execută operația SAU-EXCLUSIV între rangurile corespunzătoare operanzilor sursă și destinație.

Operație efectuată:

$$D \leftarrow S \oplus D$$

Indicatori afectați:

$$SF, ZF, PF, CF \leftarrow 0, OF \leftarrow 0.$$

DESTINAȚIE	SURSĂ
registru	registru
registru	memorie
memorie	registru
registru	imediat
memorie	imediat
acumulator	imediat

4. NOT D (Not)

Instrucțiunea execută negația logică asupra unui operand dintr-un registru sau dintr-o locație de memorie.

$$\text{Operație efectuată: } \bar{D} \leftarrow D$$

D poate fi:

- registru;
- locație de memorie.

5. TEST D, S (Test)

Instrucțiunea execută operația SI logic între rangurile corespunzătoare operanzilor sursă și destinație, dar singurul rezultat este poziționarea indicatorilor în conformitate cu rezultatul. Cei doi operanzi rămân nemodificați.

Operații efectuate:

$$D \& S;$$

$$CF \leftarrow 0;$$

$$OF \leftarrow 0.$$

Indicatori afectați:

$$CF, OF, PF, SF, ZF.$$

DESTINAȚIE	SURSĂ
registru	registru
registru	memorie
acumulator	imediat
registru	imediat
memorie	imediat

INSTRUCȚIUNI DE DEPLASARE

Aceste instrucțiuni permit deplasarea la stânga sau la dreapta a conținutului unui registru pe 8 sau 16 biți sau a conținutului unei locații de memorie, pe 8 sau 16 biți, numărul de ranguri cu care se face deplasarea putându-se specifica în două moduri:

- fie direct, caz în care deplasarea se face cu un rang;
- fie indirect, caz în care deplasarea se face cu numărul de ranguri dat de conținutul registrului CL.

6. SHL / SAL D , 1 sau CL (Shift Logical Left / Shift Arithmetic Left)

Sunt două instrucțiuni cu același efect. Se deplasează operandul (octet sau cuvânt) la stânga cu un rang sau CL ranguri; cel mai semnificativ intră în CF; în pozițiile libere se introduce zero.

D = destinație (registru sau locație de memorie).

Indicatori afectați: OF, CF.

SAL/SHF OPERANZI	EXEMPLE DE INSTRUCȚIUNI
registru , n	SAL AH , 5
memorie , n	SAL [BX].OVERDRAW , 5
registru , 1	SAL AH , 1
registru , CL	SHL DI , CL
memorie , 1	SHL [BX].OVERDRAW , 1
memorie , CL	SAL STORE_COUNT , CL

7. SHR D , 1 sau CL (Shift Logical Right)

Este o instrucțiune de deplasare logică a operandului la dreapta. Rangul cel mai puțin semnificativ va fi deplasat în indicatorul CF, iar în rangurile cele mai semnificative rămase libere, se va însera zero .

Indicatori afectați: CF, OF, PF, SF, ZF.

SHR OPERANZI	EXEMPLE DE INSTRUCȚIUNI
registru , n	SHR SI , 5
memorie , n	SHR ID_BYTE[SI][BX] , 5
registru , 1	SHR SI , 1
registru , CL	SHR SI , CL
memorie , 1	SHR ID_BYTE[SI][BX] , 1
memorie , CL	SHR INPUT_WORD , CL

8. SAR D , 1 sau CL (Shift Arithmetic Right)

Este o instrucțiune aritmetică de deplasare a operandului la dreapta. Rangul cel mai puțin semnificativ va fi deplasat în indicatorul CF, iar în rangurile cele mai semnificative rămase libere se va multiplica rangul de semn.

Dacă deplasarea se va face pe mai multe ranguri, ultima deplasare va transfera în indicatorul CF conținutul celui rang care a devenit cel mai puțin semnificativ în urma deplasărilor anterioare.

Indicatori afectați: OF, SF, ZF, PF, CF.

INSTRUCȚIUNI DE ROTATIE

Instrucțiunile de rotație acționează în mod asemănător cu cele de deplasare, cu singura deosebire că informația pierdută în cazul deplasărilor este încărcată la celălalt capăt al operandului.

9. ROL D , 1 sau CL (Rotate Left)

Instrucțiunea execută o rotație la stânga pe octet sau cuvânt cu 1 sau CL ranguri; rangul cel mai semnificativ intră și în CF .

Indicatori afectați: CF, OF.

ROL OPERANZI	EXEMPLE DE INSTRUCȚIUNI
registru , n	ROL BX , 5
memorie , n	ROL FLAG_BYTE[DI] , 5
registru , 1	ROL BX , 1
registru , CL	ROL DI , CL
memorie , 1	ROL FLAG_BYTE[DI] , 1
memorie , CL	ROL ALPHA , CL

10. ROR D , 1 sau CL (Rotate Right)

Instrucțiunea execută o rotație la dreapta pe octet sau cuvânt, cu 1 sau CL ranguri; rangul cel mai puțin semnificativ intră și în CF.

Indicatori afectați: CF, OF.

ROR OPERANZI	EXEMPLE DE INSTRUCTIUNI
registru , n	ROR AL , 5
memorie , n	ROR PORT_STATUS , 5
registru , 1	ROR AL , 1
registru , CL	ROR BX , CL
memorie , 1	ROR PORT_STATUS , 1
memorie , CL	ROR CMD_WORD , CL

11. RCL D , 1 sau CL (Rotate Left Through Carry)

Instrucțiunea execută o rotație la stânga prin Carry (CF), pe octet sau cuvânt, cu 1 sau CL ranguri; la operand se mai adaugă pe poziția cea mai semnificativă indicatorul CF, astfel încât rotația afectează fie 9, fie 17 biți.

Indicatori afectați: CF, OF.

RCL OPERANZI	EXEMPLE DE INSTRUCTIUNI
registru , n	RCL CX , 5
memorie , n	RCL ALPHA , 5
registru , 1	RCL CX , 1
registru , CL	RCL AL , CL
memorie , 1	RCL ALPHA , 1
memorie , CL	RCL [BP].PARAM , CL

RCR D , 1 sau CL (Rotate Right Through Carry)

Instrucțiunea execută o rotație la dreapta prin Carry (CF), pe octet sau cuvânt, cu 1 sau CL ranguri; la operand se mai adaugă pe poziția mai puțin semnificativă, indicatorul CF, astfel încât rotația afectează fie 9, fie 17 biți. Indicatori afectați: CF, OF.

RCR OPERANZI	EXEMPLE DE INSTRUCTIUNI
registru , n	RCR BX , 5
memorie , n	RCR [BX].STATUS , 5
registru , 1	RCR BX , 1
registru , CL	RCR BL , CL
memorie , 1	RCR [BX].STATUS , 1
memorie , CL	RCR ARRAY[DI] , CL

INSTRUCȚIUNI PENTRU LUCRUL CU SUBRUTINELE

13. CALL OP (Call)

Instrucțiunea apelează o subrutină care începe la adresa indicată de operandul OP. Dacă subrutina se află în segmentul în care se află și instrucțiunea CALL, apelul este de tip intrasegment, iar dacă subrutina se află în alt segment decât cel în care se află instrucțiunea CALL, apelul este intersegment.

OP poate fi specificat ca un operand imediat pe cuvânt, caz în care apelul se face relativ la adresa instrucțiunii CALL, intrasegment, OP reprezentând deplasamentul față de adresa instrucțiunii CALL. Valoarea maximă a deplasamentului este ± 32 octeți.

Apelul poate fi executat și indirect, dacă OP specifică un registru pe 16 biți sau adresa unei locații de memorie. Conținutul registrului respectiv sau a locației de memorie specificată și a următoarea reprezintă deplasamentul adresei față de registrul CS, de unde începe subrutina de tratare. Gradul de indirectare poate fi mărit, OP poate indica un registru care va juca rol de registru indicator. Conținutul său va fi interpretat ca o adresă. Conținutul locației respective și a următoarea reprezintă deplasamentul adresei de început a subrutinei, în raport cu registrul CS.

Din modurile de specificare ale operandului OP rezultă că apelul se va face intrasegment. În toate cazurile, înainte de execuția apelului, conținutul registrului IP, care indică adresa următoarei instrucțiuni după CALL va fi salvat în stivă, iar deplasamentul obținut prin diferitele moduri de specificare ale lui OP se va încărca în IP.

Dacă OP este specificat ca un operand imediat pe dublu cuvânt, apelul se va executa intersegment la adresa indicată de OP.

Se poate executa și un apel intersegment indirect atunci când OP indică adresa unei locații de memorie. Conținutul locației de memorie adresate și a următoarea se va încărca în IP, iar cel al următoarelor două locații de memorie se va încărca în CS. Vechile conținuturi ale regiștrilor CS și IP vor fi depuse în această ordine, în stivă.

OP poate fi:

- cuvânt (apel intrasegment);
- dublu cuvânt (apel intrasegment);
- registru 16 biți (apel intrasegment);
- adresa cuvânt (apel intrasegment);
- adresa dublu cuvânt (apel intrasegment).

14. RET sau RET OP (Return)

Este instrucțiunea de revenire din subrutina, prin refacerea din stiva a conținuturilor regiștrilor CS și IP sau numai IP, depuse de instrucțiunea CALL. Prin restaurarea acestor conținuturi se reface adresa următoarei instrucțiuni de executat de după CALL.

Îndrumar de laborator 7

Dacă există OP, el reprezintă un deplasament pe 16 biți care este adunat la conținutul registrului indicator de stivă SP, după ce au fost refăcuți regiștrii CS și IP. Astfel se pot prelua din stivă parametrii depuși înainte de execuția instrucțiunii CALL.

INSTRUCȚIUNI DE INTRARE - IEȘIRE

Sunt instrucțiuni care asigură transferul de date între registrul acumulator și un port pe 8 sau 16 biți. Specificarea portului se poate face direct, caz în care adresa sa este un octet ce urmează codului instrucțiunii, adresa portului fiind în zona $00 \div FF_{[H]}$, sau indirect, caz în care adresa portului este considerată ca fiind conținutul registrului DX și se află în spațiul de intrare/ieșire $0000 \div FFFF_{[H]}$, suprapus peste primul segment de memorie. Deci instrucțiunile de intrare/ieșire directe pot adresa maxim 256 porturi, iar cele indirecte pot adresa maxim 64 Kocteti porturi.

15. IN AL sau AX , adresa port (Input)

Instrucțiunea transferă un octet sau cuvânt, de la portul cu adresă specificată de operandul de după codul instrucțiunii, în acumulator. Dacă transferul se face pe octet, acesta poate ocupa jumătatea inferioară sau superioară a magistralei de date.

Operații efectuate: - $AL \leftarrow$ (adresa port) sau
- $AX \leftarrow$ (adresa port).

16. IN AL sau AX , DX (Input)

Este varianta indirectă a instrucțiunii de intrare IN. Adresa portului este considerată ca fiind conținutul registrului DX.

Operații efectuate: - $AL \leftarrow$ (DX) sau
- $AX \leftarrow$ (DX)

17. OUT adresa port , AL sau AX (Out)

Instrucțiunea transferă conținutul registrului AL sau AX, la portul cu adresa specificată în operandul de după codul instrucțiunii. Dacă transferul se face pe octet, acesta poate ocupa jumătatea inferioară sau superioară a magistralei de date.

Operații efectuate: - Adresa port $\leftarrow$ AL sau
- Adresa port $\leftarrow$ AX

18. OUT DX , AL sau AX (Out)

Este varianta indirectă a instrucțiunii de ieșire OUT. Adresa portului este considerată ca fiind conținutul registrului DX.

Operații efectuate: – $(DX) \leftarrow AL$ sau
– $(DX) \leftarrow AX$

INSTRUCȚIUNI PENTRU ITERAȚII

Aceste instrucțiuni sunt utile pentru implementarea buclilor. Ele utilizează conținutul registrului CX pentru a indica de câte ori se va executa o buclă. Transferul se face relativ la adresa instrucțiunii de comandă, într-o zonă ce cuprinde de la – 128 până la + 127 octeți în jurul acesteia.

19. LOOP OP (Loop)

Instrucțiunea funcționează ca o instrucțiune de salt condiționat. Ea decrementează conținutul registrului CX cu o unitate, apoi verifică dacă conținutul registrului este zero. Dacă da, instrucțiunea LOOP este ignorată și se execută instrucțiunea următoare. Dacă nu, se execută saltul la adresa indicată de operandul OP. Acesta se obține prin adresa imediată și are lungimea de un octet.

Operații efectuate: - $CX \leftarrow CX - 1$;
- dacă $CX \neq 0$ atunci $IP \leftarrow IP + OP$.
OP = operand imediat pe 8 biți.

20. LOOPE / LOOPZ OP (Loop if Equal / Zero)

LOOPE și LOOPZ sunt două mnemonici pentru aceeași instrucțiune. Se decrementează conținutul registrului CX, fără a afecta indicatorii de condiții și dacă nu are valoarea zero iar indicatorul ZF are valoarea 1, ca urmare a execuției instrucțiunii dinaintea lui LOOPE / LOOPZ, se execută saltul la adresa indicată de OP. În caz contrar, deci dacă conținutul registrului CX este zero sau indicatorul ZF are valoarea zero, instrucțiunea LOOPE / LOOPZ este ignorată și se execută instrucțiunea următoare. OP este specificat ca și în cazul instrucțiunii LOOP.

Operații efectuate: - $CX \leftarrow CX - 1$;
- dacă $CX \neq 0$ și $ZF = 1$, atunci $IP \leftarrow IP + OP$.

21. LOOPNE / LOOPNZ OP (Loop if Not Equal / Loop if Not Zero)

Efectul instrucțiunii este similar cu cel al instrucțiunii LOOPE / LOOPZ cu deosebirea că saltul se execută dacă conținutul registrului CX nu este zero și indicatorul de condiții ZF este zero.

Îndrumar de laborator 7

Operații efectuate: - $CX \leftarrow CX - 1$;
- dacă $CX \neq 0$ și $ZF = 0$ atunci $IP \leftarrow IP + OP$

INSTRUCȚIUNI PENTRU LUCRUL CU ȘIRURI

Instrucțiunile din această categorie permit efectuarea a cinci tipuri de operații asupra șirurilor:

- transferul unui șir dintr-o zonă sursă într-o zonă destinație;
- compararea a două șiruri;
- căutarea unei anumite valori într-un șir;
- crearea unui șir format din aceiași valoare;
- încărcarea acumulatorului cu elementele unui șir.

Operațiile se fac element cu element, iar acesta poate fi un octet sau un cuvânt. În general, șirul sursă se găsește în memorie într-un segment de date, iar șirul destinație se găsește în memorie într-un segment de date suplimentar. Deci accesul la un element din șirul sursă se face utilizând regiștrii DS și SI, iar accesul la un element din șirul destinație se face utilizând regiștrii ES și DI.

Instrucțiunile ce prelucrează șirurile sunt intreruptibile după transferul fiecărui element. De asemenea, ele actualizează automat adresele operanzilor sursă și destinație. În funcție de starea indicatorului de direcție DF, după fiecare transfer conținutul regiștrilor SI și DI este fie incrementat ($DF = 0$ logic), fie decrementat ($DF = 1$ logic).

22. MOVS sau MOVSW (Move byte or Word String)

Conținutul locației de memorie a carei adresă este data de regiștrii DS și SI este transferat la locația de memorie a carei adresă este data de regiștrii ES și DI. SI și DI sunt actualizați prin incrementare/decrementare cu una (MOVS), respectiv două unități (MOVSW).

Operații efectuate: - $(ES + DI) \leftarrow (DS + SI)$;
- $SI \leftarrow SI \pm (1 \text{ sau } 2)$;
- $DI \leftarrow DI \pm (1 \text{ sau } 2)$.

23. CMPSB sau CMPSW (Compare Byte or Word String)

Este o instrucțiune de comparare. Se compară elementul din șirul destinație cu cel din șirul sursă prin scăderea primului din al doilea și poziționarea corespunzătoare a indicatorilor de condiții. Accesul la operanzi se face ca și în cazul instrucțiunii MOVS, iar la sfârșitul execuției instrucțiunii se actualizează conținutul regiștrilor SI și DI, ca și în cazul instrucțiunii MOVS.

Operații efectuate: - $(DS + SI) - (ES + SI)$;

Îndrumar de laborator 7

- $SI \leftarrow SI \pm (1 \text{ sau } 2)$;

- $DI \leftarrow DI \pm (1 \text{ sau } 2)$.

Indicatori afectați:

OF, CF, PF, AF, ZF, SF.

24. SCASB sau SCASW (Scan Byte or Word String)

Este un caz particular al instrucțiunii CMPS, în care operandul sursă este conținutul registrului AL (SCASB) sau AX (SCASW). Utilizând această instrucțiune, în conjuncție cu unul din prefixele de repetare, se poate căuta o anumită valoare din cadrul unui șir sursă. La sfârșitul execuției instrucțiunii, registrul DI este actualizat.

Operații efectuate: - $AL \text{ sau } AX \leftarrow (ES + DI)$; Indicatori afectați:

- $DI \leftarrow DI \pm (1 \text{ sau } 2)$. OF, CF, PF, AF, ZF, SF.

25. LODSB sau LODSW (Load Byte or Word String)

Este o instrucțiune de transfer. Operandul sursă, a cărui adresă se obține din regiștrii DS și SI este încărcat în regiștrii AL (LODSB) sau AX (LODSW). Transferul se face pe octet sau cuvânt, iar la sfârșitul execuției instrucțiunii, registrul SI este actualizat.

Operații efectuate: - $AL \text{ sau } AX \leftarrow (DS + SI)$

26. STOSB sau STOSW (State Byte or Word String)

Este tot o instrucțiune de transfer. Conținutul registrului AL (STOSB) sau AX (STOSW) este transferat în memorie la adresa indicată de regiștrii ES și DI, după care conținutul registrului DI este actualizat. Utilizând și un prefix pentru repetare, cu ajutorul acestei instrucțiuni se poate crea în memorie un șir format din același element.

Operații efectuate: - $(ES + DI) \leftarrow AL \text{ sau } AX$;

- $DI \leftarrow DI + 1 \text{ sau } 2$.

27. REP (Repeat)

Este un prefix care asigură repetarea hardware a instrucțiunii pe care o precedă. Se utilizează în conjuncție cu una din instrucțiunile MOVS sau STOS și efectul sau este următorul: se testează conținutul registrului CX; dacă este zero se execută instrucțiunea următoare, iar în caz contrar CX este decrementat cu o unitate și apoi se execută instrucțiunea care are ca prefix pe REP, după care se reia procesul. Poziționând registrul CX cu valoarea dorită se asigură repetarea prin hardware a anumitor operații. Astfel instrucțiunea REPMOVS va asigura transferul unui șir dintr-o zonă sursă într-o zonă destinație, iar REPSTOSB va

Îndrumar de laborator 7

determina crearea unui șir format din octeți cu aceiași valoare. Lungimea șirului prelucrat este fixată de conținutul registrului CX.

28. REPE / REPZ (Repeat While Equal / Zero)

Este un prefix care determină repetarea unei instrucțiuni, dacă conținutul registrului CX este diferit de zero și indicatorul ZF are valoarea "1" logic. Se utilizează în conjuncție cu una din instrucțiunile CMPS sau SCAS și are efectul de comparare, respectiv de căutare repetată. Cele două condiții au semnificațiile:

- conținutul registrului CX nu este zero: înseamnă că operația se repetă până când șirul se termină;

- indicatorul ZF are valoarea 1: înseamnă că operația se repetă dacă elementele corespunzătoare din cele două șiruri sunt egale.

Instrucțiunea REPESCASB poate fi folosită pentru căutarea într-un șir a unei valori diferite de o valoare fixă, dată în acumulator.

29. REPNE / REPNZ (Repeat While Not Equal / Not Zero)

Operează analog cu prefixul REPE / REPZ, cu aceleași instrucțiuni, cu deosebirea că repetarea se face dacă conținutul registrului CX nu este zero, deci șirul nu s-a încheiat și indicatorul ZF are valoarea zero logic, adică elementele care se compară nu sunt egale. Repetarea se încheie dacă toate elementele șirului au fost comparate sau la prima egalitate întâlnită.

Instrucțiunea REPNECASCASB poate fi folosită pentru căutarea într-un șir a unei valori egale cu o valoare fixată în acumulator.

INSTRUCȚIUNI DE CONTROL

În această categorie au fost cuprinse instrucțiuni care acționează asupra indicatorilor de condiții, instrucțiuni pentru sincronizarea microprocesorului cu evenimente externe și o instrucțiune al cărei efect este ca microprocesorul nu execută nici o operație.

30. CLC (Clear Carry)

Instrucțiunea șterge indicatorul CF. Noua valoare a indicatorului va fi "0" logic indiferent de valoarea dinaintea execuției instrucțiunii.

Operație efectuată: $CF \leftarrow 0$; Indicatori afectați: CF.

31. CMC (Complement Carry)

Efectul instrucțiunii este negarea conținutului indicatorului CF.

Operație efectuată: $CF \leftarrow \overline{CF}$ Indicatori afectați: CF.

32. STC (Set Carry)

Operație efectuată: $CF \leftarrow 1$; Indicatori afectați: CF.

33. CLD (Clear Direction)

Instrucțiunea poziționează indicatorul de direcție în “0” logic, ceea ce va determina autoincrementarea regiștrilor DI și SI în cazul instrucțiunilor care lucrează cu șiruri.

Operație efectuată: $DF \leftarrow 0$; Indicatori afectați: DF.

34. STD (Set Direction)

Instrucțiunea poziționează indicatorul de direcție în “1” logic, ceea ce va determina autodecrementarea regiștrilor DI și SI în cazul instrucțiunilor care lucrează cu șiruri.

Operație efectuată: $DF \leftarrow 1$; Indicatori afectați: DF.

35. CLI (Clear Interrupt)

Instrucțiunea anulează conținutul indicatorului de întreruperi IF, ceea ce va duce la dezactivarea sistemului de întreruperi mascabile; cererile de întrerupere externe, ce sosesc pe linia INTR nu vor fi luate în considerare.

Operație efectuată: $IF \leftarrow 0$; Indicatori afectați: IF.

36. STI (Set Interrupt)

Instrucțiunea poziționează în “1” logic, indicatorul de întreruperi IF, ceea ce va duce la activarea sistemului de întreruperi; cererile de întrerupere externe, ce sosesc pe linia INTR vor fi luate în considerare.

Operație efectuată: $IF \leftarrow 1$; Indicatori afectați: IF.

37. HLT (Halt)

În urma execuției acestei instrucțiuni microprocesorul intră într-o stare de oprire (halt) pe care o va părăsi doar la apariția semnalului de inițializare RESET, a unei cereri de întrerupere nemascabile sau a unei cereri de întrerupere mascabile, dacă sistemul de întreruperi este activat.

38. WAIT (Wait)

Este o instrucțiune ce se poate folosi la sincronizarea microprocesorului cu evenimente externe. Efectul ei este intrarea microprocesorului în starea de așteptare, dacă linia TEST este inactivă.

Îndrumar de laborator 7

Activarea liniei TEST determină reluarea activității normale.

39. LOCK (Lock Bus)

Este un prefix pe un octet care are ca efect activarea de către microprocesor (în modul de lucru maximal) a liniei LOCK în timpul execuției instrucțiunii precedată de acest prefix.

40. ESC COD , OP (Escape)

Este o instrucțiune care permite unui procesor extern să preia de la microprocesorul 8086 un cod de operație și, eventual, un operand din memorie. COD este un cod de operație pe șase ranguri care intră în codul mașină al instrucțiunii ESC și care este preluat de procesorul extern la extragerea acesteia. Dacă OP indică un registru, microprocesorul nu efectuează nici o operație, iar dacă OP indică o locație de memorie, microprocesorul execută un ciclu de citire, dar data va fi preluată de procesorul extern.

ESC Operanzi	Exemplu
imediat , memorie	ESC 6 , ARRAY [SI]
imediat , registru	ESC 20 , AL

41. NOP (No Operation)

Procesorul nu efectuează nici o operație.

INSTRUCTIUNI PENTRU INTRERUPERI

Întreruperile permit abandonarea momentană de către microprocesor a execuției unui program și începerea execuției unui alt program situat oriunde în spațiul de memorie direct adresabil.

În cazul microprocesorului 8086 întreruperile pot fi generate fie datorită unor cauze externe microprocesorului, fie datorită unor cauze interne. Ca urmare, ele se împart în două mari categorii:

- întreruperi externe;
- întreruperi interne.

Microprocesorul 8086 poate trata 256 tipuri de întreruperi, fiecare fiindu-i asociat un cod care-l identifică. Legătura între diferitele tipuri de întreruperi și procedurile care le deservesc se face prin intermediul unei tabele a vectorilor de întrerupere situată în domeniul 00000_[H] ÷ 003FF_[H].

Fiecare vector ocupă patru locații de memorie și are următoarea structură: cel mai puțin semnificativ octet al registrului IP, cel mai semnificativ octet al registrului IP, cel mai puțin semnificativ octet al registrului CS și cel mai semnificativ octet al registrului CS. Practic, un vector conține adresa completă a subrutinei de tratare a întreruperii. Pentru a realiza accesul la un vector, microprocesorul multiplică cu patru codul pe care-l primește și care identifică tipul întreruperii.

Tabela vectorilor de întrerupere:

Zona disponibilă	003FF _[H]	TIP 255	Disponibil
	003FC _[H]	TIP 254	Disponibil
	003F8 _[H]		
Zona rezervată	00084 _[H]	TIP 33	Disponibil
	00080 _[H]	TIP 32	Disponibil
	0007F _[H]	TIP 31	Rezervat
	00018 _[H]		
Zona dedicată	00014 _[H]	TIP 5	Rezervat
	00010 _[H]	TIP 4	Depasire
	0000C _[H]	TIP 3	Breakpoint
	00008 _[H]	TIP 2	Nemascabil
	00004 _[H]	TIP 1	Pas cu pas
	00000 _[H]	TIP 0	Eroare la divizare

Tabela vectorilor de întrerupere este împărțită în trei domenii:

- o zonă dedicată, care este utilizată în mod automat de microprocesor la apariția unor tipuri de întrerupere, predefinite;
- o zonă rezervată, pentru a păstra compatibilitatea cu produsele firmei INTEL, bazate pe microprocesorul 8086 ;
- o zonă disponibilă pentru cererile de întrerupere utilizator.

Dacă într-o anumită aplicație nu sunt folosite întreruperile predefinite, utilizatorul poate folosi și zona dedicată.

Întreruperile externe

Sunt inițiate prin intermediul celor două intrări dedicate ale microprocesorului, INTR și NMI. O cerere pe linia NMI generează o întrerupere nemascabilă și este predefinită de tipul 2, adică microprocesorul execută accesul la vectorul 2 fără alte informații suplimentare.

O secvență începe prin apariția pe linia INTR a unei cereri de întrerupere. În general, ea este generată de un circuit exterior care evaluează prioritatea, în cazul apariției cererilor simultane din mai multe surse și lasă să treacă cererea cu prioritate maximă la momentul respectiv. Semnalul INTR este testat de

Îndrumar de laborator 7

microprocesor în ultima stare a ultimului ciclu la execuția unei instrucțiuni. Există excepții de la această regulă și anume, cazul instrucțiunilor care lucrează cu șiruri pentru care linia INTR este testată după prelucrarea fiecărui element al șirului și cazul instrucțiunii WAIT la care întreruperea este testată după fiecare verificare a liniei

Există și câteva situații în care cererea de întrerupere nu este luată în considerare decât după execuția următoarei instrucțiuni. Este cazul instrucțiunilor precedate de prefixe: nu se recunoaște nici o cerere de întrerupere între execuția prefixului și execuția instrucțiunii. De asemenea, este cazul instrucțiunilor MOV și POP care încarcă regiștrii segment; nu se recunoaște nici o cerere de întrerupere până când nu se execută și următoarea instrucțiune. Această măsură este una de protecție. Dacă se dorește schimbarea unui segment (de cod, de stivă, etc.) este necesară modificarea conținuturilor a doi regiștri. Dacă se permite accesul unei cereri de întrerupere după schimbarea conținutului registrului segment și înaintea schimbării conținutului celui alt registru există riscul ca acest ultim transfer să nu se mai facă corect, ceea ce va conduce la o altă locație din noul segment decât cea dorită. Din cele de mai sus rezultă că dacă se dorește schimbarea unui segment este necesară execuția mai întâi a instrucțiunii ce transferă informația în celălalt registru.

Pentru luarea în considerare a unei cereri de întrerupere ce sosește pe intrarea INTR este necesar ca sistemul de întreruperi al microprocesorului să fie activat, adică indicatorul de întreruperi IF să aibă valoarea "1" logic. Dacă IF = "0" logic, cererea de întrerupere nu va fi luată în considerare. Pentru resetarea indicatorului de întrerupere se poate utiliza instrucțiunea CLI, iar pentru setarea sa se poate folosi instrucțiunea STI.

Dacă indicatorul de întrerupere este activat, microprocesorul va lua în considerare cererea de întrerupere de pe linia INTR. Imediat, indicatorul IF va fi anulat pentru a preveni alte atenționări ale unor noi cereri de întrerupere. De asemenea, indicatorul TF va fi anulat invalidând facilitatea de lucru în modul pas cu pas. În continuare, microprocesorul depune în stivă conținutul indicatorilor (IF și TF având vechile valori) și a regiștrilor CS și IP, actualizând de fiecare dată registrul SP. Din acest moment începe ciclul extern de acceptare a cererii de întrerupere, prezentat în figura următoare, care durează doi cicli de magistrală.

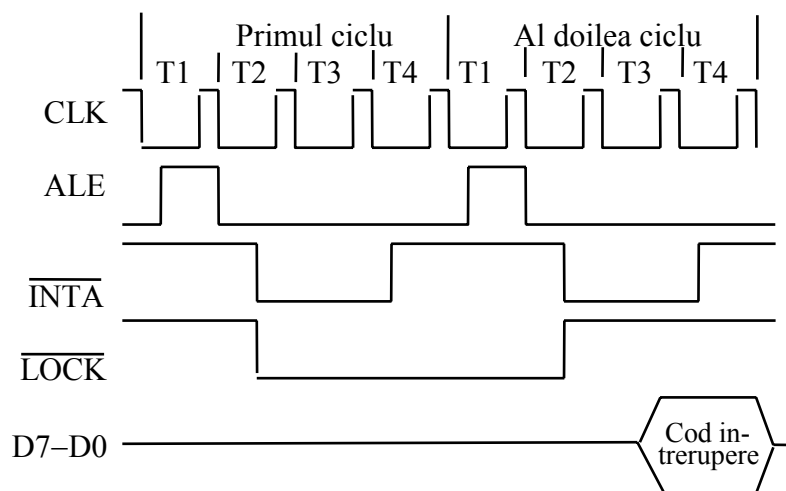
Simultan cu activarea liniei /INTA microprocesorul activează și linia /LOCK indicând exteriorului că nu poate ceda controlul magistralelor. Acest semnal va trebui folosit de circuitele externe, inclusiv arbitrul de magistrală, pentru a nu permite accesul unui alt master. Dacă microprocesorul este în mod minimal, semnalul /LOCK se va activa doar intern, dar o cerere HOLD nu va fi luată în considerare.

În al doilea ciclu de magistrală, corespunzător acceptării unei cereri de întrerupere, microprocesorul va activa din nou linia /INTA, indicând exteriorului că așteaptă codul care identifică sursa ce a generat întreruperea. În general, acest cod este furnizat de autorul cererii de întrerupere prin intermediul circuitului specializat pentru întreruperi 8259A. Microprocesorul preia codul și execută accesul la

Îndrumar de laborator 7

subrutina de tratare a întreruperii, prin intermediul tabelii vectorilor de întrerupere. În acest timp el execută două cicluri de citire cuvânt din memorie pentru a încărca regiștrii IP și CS.

Ciclu de acceptare a unei întreruperi externe



În starea T1 a primului ciclu este generat semnalul ALE, dar magistrala de adrese/date trece în a treia stare și rămâne în această stare până la sfârșitul ciclului. Apoi se activează linia /INTA indicând exteriorului că cererea de întrerupere a fost luată în considerare și practic, cererea poate fi dezactivată.

Subrutina de tratare a întreruperii va trebui să se încheie cu instrucțiunea IRET, care va realiza revenirea în programul principal și restabilirea contextului inițial înainte de luarea în considerare a cererii de întrerupere, prin citirea din stivă a conținutului registrului indicatorilor de condiții și regiștrilor CS și IP.

Utilizatorul are la dispoziție, pentru cererile de întrerupere externe zona 00080_[H] – 003FF_[H] din tabela vectorilor de întrerupere, adică dispune de maxim 224 vectori. Evident că dacă nu se dorește compatibilizarea cu celelalte produse ale firmei INTEL și nici nu se folosesc întreruperile predefinite, se pot utiliza și ceilalți vectori.

Cererea de întrerupere nemascabilă, după cum s-a afirmat, este predefinită de tipul 2. Pentru a fi luată în considerare este necesar să aibă o durată de minim două stări. Ea este de asemenea testată la sfârșitul unei instrucțiuni.

Operațiunile executate de microprocesor sunt salvarea conținutului curent al indicatorilor de condiții, salvarea regiștrilor IP și CS și încărcarea lor cu informația din tabela vectorilor de întrerupere, corespunzător vectorului 2.

Întreruperile interne constituie o altă clasă de întreruperi. Diferența față de cele externe constă în faptul că sursa de întreruperi nu este un eveniment extern, ci întreruperea apare ca urmare a execuției unei instrucțiuni tipice sau a producerii unui eveniment intern.

Ele se împart în două categorii:

- întreruperi generate de instrucțiunea INT nn;
- întreruperi predefinite.

Instrucțiunea INT nn ocupă doi octeți în memorie, codul instrucțiunii și tipul întreruperii, ce realizează legătura cu vectorul nn din tabela vectorilor de întrerupere. Deci are același efect ca și o întrerupere externă de tip nn, doar ca nu este mascabilă și microprocesorul nu execută o secvență de acceptare a ei (nu se vor genera semnale /INTA).

Operațiile executate de microprocesor la luarea în considerare a acestui tip de întrerupere sunt aceleași ca și în cazul celor externe, fără însă a se genera semnalele /INTA.

Subrutina pentru tratarea acestor întreruperi poate fi folosită pentru emulare, pentru testare, etc. De exemplu, o subrutină pentru execuția unei operații aritmetice mai complexe poate fi apelată prin intermediul unei instrucțiuni INT nn.

Întreruperile predefinite sunt generate de microprocesor, în mod automat, la detectarea apariției unor anumite evenimente interne. Aceste întreruperi sunt nemascabile iar pentru tratarea lor, microprocesorul execută aceleași operații ca și în cazul instrucțiunilor INT nn.

Întreruperile predefinite apar în urma detectării următoarelor evenimente:

- împărțirea prin zero – tip 0;

Întreruperea este generată în urma execuției instrucțiunilor de împărțire, dacă câtul este mai mare decât valoarea maximă admisă. Este considerat că făcând parte din instrucțiunea de împărțire. În cazul instrucțiunii DIV valoarea câtului care determină generarea întreruperii este $FF_{[H]}$ dacă împărțitorul este octet sau $FFFF_{[H]}$ dacă împărțitorul este cuvânt, iar în cazul instrucțiunii IDIV această valoare este $7F_{[H]}$ în primul caz și $7FFF_{[H]}$ în cel deal doilea.

- lucrul pas cu pas – tip 1;

Întreruperea apare după setarea indicatorului de control TF. La sfârșitul execuției fiecărei instrucțiuni, microprocesorul va lua în considerare o întrerupere predefinită de tip 1. Subrutina de tratare a întreruperii poate afișa conținutul regiștrilor, a memoriei sau poate oferi utilizatorului informații importante pentru verificarea, depanarea și punerea la punct a programelor.

Microprocesorul nu dispune de instrucțiuni pentru modificarea directă a conținutului indicatorului TF. Operația se poate realiza prin intermediul stivei. Instrucțiunea PUSHF depune în stivă indicatorii, aceștia putând fi modificați prin intermediul instrucțiunilor logice și reîncărcați prin instrucțiunea POPF.

- întreruperi nemascabile NMI – tip 2;

Îndrumar de laborator 7

- întreruperi de tip "breakpoint" – tip 3;

Este utilizata prin fixarea unor puncte de întrerupere (poziționarea unor octeți) pe parcursul derulării unui program în scopul testării, depanării și punerii la punct. Are avantajul de a ocupa un singur octet în memorie.

- întreruperi la depășire – tip 4;

Întreruperea este generată în urma apariției depășirii capacității regiștrilor, oricare ar fi operația aritmetică ce se execută, deci dacă indicatorul OF = "1" logic. Întreruperea nu se generează automat, ci numai dacă microprocesorul execută o instrucțiune INTO, care în general trebuie să urmeze unei instrucțiuni aritmetice având operanzi cu semn. Subrutina de tratare poate să conțină un mesaj către utilizator pentru a-l anunța de apariția depășirii.

42. INT nn

Instrucțiunea determină generarea unei întreruperi interne de tip nn; microprocesorul execută un acces la vectorul nn din tabela vectorilor de întrerupere și nu execută un ciclu extern de acceptare a cererii.

Operații efectuate: - $SP \leftarrow SP - 2$

- $(SP+1:SP) \leftarrow \text{Indicatori}$

- $TF \leftarrow 0$

- $IF \leftarrow 0$

Indicatori afectați:

IF, TF.

- $SP \leftarrow SP - 2$

- $(SP+1:SP) \leftarrow CS$

- $CS \leftarrow \text{TYPE} \times 4 + 2$

- $SP \leftarrow SP - 2$

- $(SP+1:SP) \leftarrow IP$

- $IP \leftarrow \text{TYPE} \times 4$

43. INTO

Instrucțiunea determină generarea unei întreruperi predefinite de tip 4, în cazul în care indicatorul de depășire OF = "1" logic, în urma execuției unei instrucțiuni aritmetice având operanzi cu semn.

44. IRET

Instrucțiunea determină revenirea în programul principal, prin restaurarea conținutului registrului indicatorilor de condiții și a regiștrilor IP și CS cu informația citită din stivă.

Considerații comune tuturor tipurilor de întrerupere

Timpul de procesare al unei întreruperi, definit din momentul în care cererea este recunoscută și până când microprocesorul execută accesul la adresa de început a subrutinei de tratare, depinde de tipul întreruperii.

Pentru a determina timpul de acceptare a cererii de întrerupere în cazul celor externe, trebuie luat în considerare și momentul apariției în raport cu execuția instrucțiunii. S-a stabilit că timpul cel mai lung de așteptare se obține atunci când cererea de întrerupere apare în timpul execuției unei instrucțiuni de înmulțire, împărțire sau deplasare și rotire cu un număr variabil de ranguri.

Timpul de procesare al întreruperilor la microprocesorul 8086:

TIP DE ÎNTRERUPERE	TIMP ÎN PERIOADE DE TACT
Întrerupere externă (INTR)	61
Întrerupere nemascabilă (NMI)	50
INT nn	51
INT 3	52
INTO	53
INT1	50

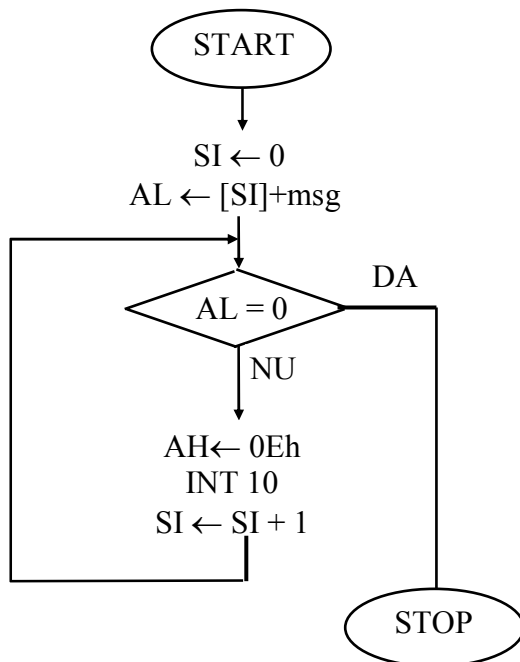
O altă problemă comună se referă la prioritatea cu care microprocesorul tratează cererile de întrerupere în cazul apariției simultane.

Gradul de prioritate a diferitelor tipuri de întreruperi la microprocesorul 8086:

TIPUL DE ÎNTRERUPERE	PRIORITATE
Împărțire la zero, INT nn	Maximă
INTO	
NMI	
INTR	
Pas cu pas	Minimă

Probleme

1. Să se realizeze un program (executabil) care să afișeze pe monitorul PC șirul de caractere „Laborator microprocesoare”. Afișarea pe ecran se face prin executarea unei întreruperi interne (BIOS) și anume întreruperea 10. La executarea acestei întreruperi registrul AH trebuie să conțină valoarea 0Eh iar registrul AL caracterul care va fi afișat.



```

#make_COM#
ORG 100h
    JMP start
msg DB 'Facultatea de
      electronica si electromecanica',
      0
start:
    MOV SI, 0
next_char:
    MOV AL, [SI]+msg
    CMP AL, 0
    JE stop
    MOV AH, 0Eh
    INT 10h
    INC SI
    JMP next_char
stop:
    RET
END
  
```

În registrul SI se va încărca deplasamentul față de adresa de bază unde se află șirul de caractere. La apelarea întreruperii interne 10h procesorul va suspenda execuția buclei principale, va salva indicatorul și adresa buclei principale iar apoi va fi citit conținutul registrului AX (AL și AH). În AH se află codul 0Eh pentru transfer în memoria video iar în AL codul ASCII al caracterului care va fi afișat. Programul se va termina când se va citi valoarea 0h, aceasta reprezintă caracterul terminator. Această procedură este folosită la majoritatea limbajelor de programare de nivel înalt, orice șir de caractere este urmat de caracterul nul (0h, în C de exemplu reprezentat ‘\0’) care indică terminarea acestui șir.

De rezolvat:

- Să se realizeze un program executabil în care să afișeze pe ecran tot setul de caractere ASCII începând cu cifre, litere mari, semne punctuație, litere mici.
- Să se realizeze un program executabil care să afișeze pe ecran literele mari în formatul: litera-cod ASCII hexa- cod ASCII zecimal.

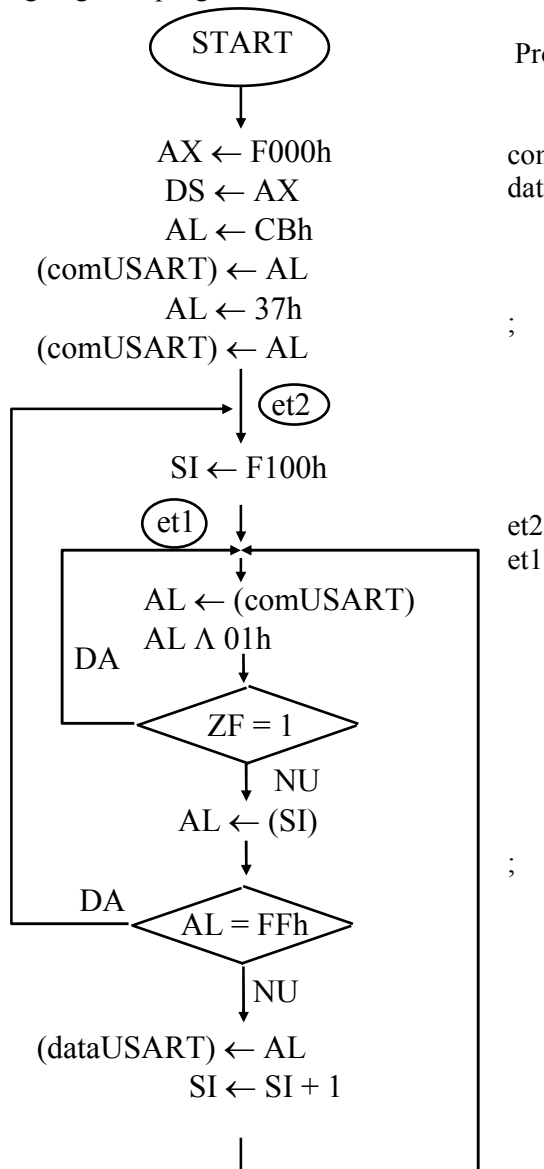
2. În memoria ROM a unui microsistem cu procesor 8086, la adresa FF100h se află plasat un text (în cod ASCII). Caracterul terminator al textului este FFh. Se cere realizarea unui program în vederea tipăririi acestui text la video-terminalul conectat serial la microsistem. Călea serială a microsistemului este

Îndrumar de laborator 7

gestionată de un circuit USART (8251) a cărei adresa de comenzi este 82h și adresa de date 80h. Tipărirea trebuie să se realizeze cu reluare după finalizarea tipăririi anterioare.

Soluție

Organigrama programului este:



Programul in limbaj de asamblare:

```

org fff0h
comusart equ 82h
datausart equ 80h
mov ax,f000h
mov ds,ax
jmp fff0h:0000h

;

org f000h
mov al,cbh
out comusart,al
mov al,37h
out comusart,al
mov si,f100h
in al,com
test al,01h
jz et1
mov al,[si]
cmp al,ffh
jz et2
out datausart,al
inc si
jmp et1

;

org f100h
str 'Acest text provine din
memoria ROM adresa
ff100 !'
db 0dh,0ah,ffh
  
```

Pentru adresarea șirului de date, reprezentat prin textul deșus în memorie începând cu adresa FF100h, se folosește registrul segment DS, inițializat cu valoarea F000h. Propriu-zis, șirul de date va fi adresat prin intermediul registrului

Îndrumar de laborator 7

SI, inițializat cu valoarea 100h. Pentru lucrul cu circuitul USART este nevoie de programarea acestuia, efectuată prin trimerile octeților CBh și 37h, pe adresa de comenzi. Studiul circuitului USART se va realiza în cadrul cursului 'Sisteme Electronice Programabile'. Circuitul USART gestionează comunicațiile seriale în cadrul unui microsistem de calcul. El serializează datele oferite în format paralel de procesor, în momentele de transmisie și le deserializează la recepție pentru preluarea paralelă de către procesor. Transmisia serială demarează după înscrierea codului caracterului ce urmează a fi transmis la adresa de date a USART-ului. Totuși, momentele în care transmisia se poate efectua sunt semnalizate într-un registru de stare accesibil prin citire de la adresa de comenzi. Semnalizarea se efectuează prin primul bit din acest registru, care în starea "1"- logic, indică posibilitatea efectuării transmisiei. Programul realizat efectuează testul acestui bit, permițând încărcarea caracterului de transmis doar când starea este "1"- logic. Caracterul preluat din memorie este supus unui test prin care se constata dacă acesta reprezintă un terminator (FFh). Dacă da, întreg programul de tipărire este reluat de la primul caracter. Altfel, situația este de tipărire a unui caracter din cadrul secvenței, caz în care se continuă normal. Pentru accesarea următorului caracter, registrul SI este incrementat.

De rezolvat:

- a. Un șir de date format din 100 octeți este plasat în memoria unui microsistem cu microprocesor 8086, începând cu adresa F0000h. Se cere realizarea unui program pentru transferarea acestui șir, păstrând ordinea termenilor, într-o altă zonă de memorie de tip RAM, ce începe cu adresa D1000h, în spațiul aceluiași microsistem.
- b. Modificați problema anterioară pentru ca plasarea termenilor în zona de memorie RAM să se facă în ordine descrescătoare din punct de vedere al adreselor șirului sursă.
- c. Se da un șir format din 200 octeți plasat în memoria unui microsistem cu microprocesor 8086, începând cu adresa C0000h. Termenii șirului sunt diferiți. Se cere realizarea unui program pentru aflarea minimului acestor termeni. Minimul va fi plasat în final în registrul BL.